AI Agent Skills and MCP: Building the Right AI Agent Pattern

Stephen Simon

Chapter 3: Building AI Workflows with MCP

Connecting an Agent to MCP Tools and Resources

Connect Claude Code to the Orbit Labs MCP server and explore what it exposes

At the end of Chapter 2, the `launch-risk-review` Skill couldn't list current support tickets, couldn't see what changed after the last document, and couldn't record anything in the team's risk register. Those are access problems. In this video you connect your agent to Orbit Labs' company systems through an MCP server and look at the tools and resources it provides.

Before you start

Your workspace should be at the end state of Chapter 2. If you're not sure, run this from the folder that contains `orbit-launch` and `solutions`:

```
node solutions/catch-up.mjs 02_01
```

Keep `reports/skill-only.md` from video 02_04 if you saved it. You'll use it in video 03_03.

What's already in the project

- `orbit-backend/server.js` is a prebuilt MCP server. It stands in for Orbit Labs' project tracker and beta support desk. You won't write or change it in this course.
- `orbit-backend/seed/` holds the data behind it: milestones, tasks, beta support tickets, and an empty risk register.
- `orbit-launch/.claude/skills/launch-risk-review/` is your Skill from Chapter 2.

Step 1: Set the Skill aside for this chapter

This chapter shows what MCP does on its own, so the agent shouldn't load the Skill. Testing one change at a time makes it clear which part of the result comes from which piece. You'll bring the Skill back in Chapter 4.

From a terminal in `orbit-launch`, move the Skill into a `parked-skills` folder. On macOS or Linux:

```
mkdir -p parked-skills
mv .claude/skills/launch-risk-review parked-skills/
```

On Windows (PowerShell):

```
New-Item -ItemType Directory -Force parked-skills
Move-Item .claude\skills\launch-risk-review parked-skills\
```

Step 2: Check the server

From a terminal in `orbit-backend`, run:

```
npm run check
```

You should see seven tools, two resources, and `Open tickets older than 7 days: 6`. If you see an error, run `npm install` first.

Step 3: Register the server with Claude Code

From a terminal in `orbit-launch`, run:

```
claude mcp add --scope project orbit -- node ../orbit-backend/server.js
```

Here's what each part means:

- `orbit` is the name the agent will use for this server.
- `--scope project` saves the connection in the project, so everyone who opens this workspace gets it.
- Everything after `--` is the command that starts the server. Claude Code runs it as a local process and talks to it over standard input and output.

The command creates a `.mcp.json` file in `orbit-launch`:

```
{
  "mcpServers": {
    "orbit": {
      "type": "stdio",
      "command": "node",
      "args": ["../orbit-backend/server.js"]
    }
  }
}
```

You can commit this file to your repository so the whole team shares the same connection, just like the Skill.

Using a different agent? Most agents accept the same information: a server name, a command, and its arguments. Check your agent's documentation for where its MCP configuration lives. If it doesn't start from the `orbit-launch` folder, use the absolute path to `server.js`.

Step 4: Start a session and confirm the connection

1. Start a new Claude Code session in `orbit-launch`. The first time, Claude Code asks whether to use the project's MCP server. Approve `orbit`.

2. Type `/mcp`. You should see `orbit` listed as connected.

Step 5: Ask what the server provides

Send:

```
What tools and resources does the orbit server give you?
```

You should see seven tools and two resources:

- **Tools:** `get_project`, `list_milestones`, `list_tasks`, `search_tickets`, `get_ticket`, `list_risk_items`, and `create_risk_item`.
- **Resources:** `orbit://project` and `orbit://milestones`.

Step 6: Make the first tool call

Send:

```
What's the Orbit 2.0 launch date, and what date is this review for?
```

The agent asks for permission to call `get_project`. Approve it. The answer should be **November 16, 2026** for the launch and **October 5, 2026** for the review date.

Why a fixed review date? The dataset measures ticket ages against October 5, 2026, not today's date. That way every learner sees the same results, whenever they take the course.

Tools and resources

An MCP server can expose several kinds of capabilities. The two you'll use in this course are:

- **Tools** are functions the model decides to call, like `search_tickets` or `create_risk_item`. They can read data or take actions.
- **Resources** are read-only data the application can attach to the conversation as context, like `orbit://milestones`. They're usually chosen by you rather than by the model. In Claude Code, you can attach one by typing `@` and choosing it from the list.

How the agent knows which tool to use

Here's the definition of `search_tickets` from `server.js`, shortened slightly:

```
server.registerTool(
  "search_tickets",
  {
    title: "Search beta support tickets",
    description:
      "Searches beta support tickets. Each result includes daysOpen,
measured against the project review date. Filter by status, priority, tag,
or minimum days open.",
    inputSchema: {
      status: z.enum(["open", "pending_customer", "closed"]).optional(),
      priority: z.enum(["P1", "P2", "P3", "P4"]).optional(),
```

```
      tag: z.string().optional().describe("Ticket tag, for example
payments or auth."),
      min_days_open: z.number().int().min(0).optional(),
    },
    annotations: { readOnlyHint: true },
  },
  async (args) => { /* reads the support desk data */ }
);
```

The agent never sees the code that runs the tool. It sees the name, the description, and the input schema. Those are what it uses to decide which tool fits a request and which arguments to pass. It's the same idea as a Skill's description, applied to a single capability.

Notice what the description says: what the tool does and what it returns. It doesn't say anything about how Orbit Labs judges launch risk. That's a different kind of knowledge, and it will matter later in this chapter.

Reading tools and writing tools

Six of the seven tools only read data. `create_risk_item` writes to the shared risk register that the whole team sees. The server marks each tool with a hint about whether it's read-only, and Claude Code asks before calling tools. Pay attention to which calls you approve, especially for tools that change data.

If you get stuck

From the folder that contains `orbit-launch` and `solutions`, run:

```
node solutions/catch-up.mjs 03_01
```

This parks the Skill and creates `.mcp.json`. Start a new Claude Code session afterward and approve the server.